A Portable Finite-State Architecture for Deterministic Checkers Play

Christopher A. Tucker
cartheur@pm.me

Draft — November 2026

Abstract

This paper introduces a work-in-progress architecture for a deterministic Checkers player designed to run across constrained and desktop environments. The controller is expressed as a finite-state machine (FSA) with explicit request, validation, rule generation, bounded projection, evaluation, selection, response, and error states. The design separates the control machine from policy: the FSA governs safe, inspectable progress, while a replaceable policy ranks legal candidate moves. A portable C++11 core, a word-oriented mailbox protocol, board-geometry fixtures, and initial men-only rules fixtures provide the current implementation evidence. This draft reports the architecture and its validation approach; it does not yet claim game-playing strength, complete Checkers rules, or novelty relative to prior game programs.

1 Introduction

Classic game systems demonstrate that useful computer players need not rely on large learned models. This project investigates a different question: can a small, deterministic controller provide a transparent and portable foundation for Checkers play across a host emulator and constrained target hardware?

The proposed answer is an FSA with explicit state, bounded work, and observable mailbox results. It is intended to make each stage inspectable and fixture-testable rather than embedding control flow implicitly in a monolithic search routine.

2 Architecture

The controller begins in q_0 = WAIT_REQUEST. It snapshots and validates an incoming board, reconciles prior results, generates legal moves, and enters q_4 = ANTICIPATE. In that state it performs bounded projection of candidate lines before evaluation and selection. A retained move enters q_7 = DECISION, whose transition writes the successful response; invalid or terminal work enters q_E = ERROR, whose transition writes the error response. The current state diagram and formal input/output alphabets are maintained with the source repository.

Current evidence artifact: `research/checkers-evidence/FSA-LOGIC.md`. The rendered diagram is `FSA-LOGIC.png`. At this draft stage the portable core implements the request/response skeleton; move generation provides evidence for the next generation state.

2.1 Separation of control and policy

The FSA is not the game strategy. It defines state transitions, memory ownership, failure paths, and commit boundaries. In particular, `ANTICIPATE` is a controller state, while projection is the bounded operation it performs; `DECISION` and `ERROR` name conditions, while their outgoing transitions perform mailbox writes. Policy is a separate, deterministic ranking function over legal projected moves. This separation permits policy experiments without weakening protocol or rules validation.

3 Anticipation and decision

The central distinction in this architecture is between anticipating a consequence and deciding what to expose to the game surface. The distinction is useful even for a small deterministic player because it places board simulation, policy, memory, and external effects on different sides of an explicit boundary.

3.1 `ANTICIPATE`: bounded projection, not commitment

The `ANTICIPATE` state receives a legal candidate from generation and constructs a private projected position. At minimum, projection applies the candidate move and records captures, promotion, turn ownership, and any required jump continuation. A later implementation may extend the same state to project a bounded opponent reply and a reply-to-reply line. Thus, anticipation is not a prediction that the opponent must make a particular move; it is a finite, reproducible construction of positions that may result from a candidate line.

Three constraints make this state suitable for a constrained system:

1. **Legality precedes projection.** The state consumes only a generated legal move; it does not invent board transitions or bypass the rules engine.

2. **Projection is private and bounded.** The accepted board remains immutable while a separate scratch position and transition trace are built. Each FSA invocation performs a bounded unit of work, such as applying one candidate or one continuation, before yielding control.

3. **Projection remains inspectable.** The trace retains more than a numerical value: it identifies the candidate, projected board, captures, continuations, and later policy value. It can therefore explain why a selected move was possible and support reconciliation with the next observed board.

The state may loop while further candidate lines need projection. Its exit condition is not a likely move being found, but that the explicitly bounded set of requested projections is complete. This gives the policy a defined input and prevents a search-like task from becoming an unbounded bus or frame latency source.

3.2 `DECISION`: selection becomes an accountable boundary

`DECISION` begins only after evaluation has assigned values to the complete candidate set and `SELECT` has retained one candidate using a deterministic ordering. It is therefore not another evaluation pass. Its responsibility is to stabilize the selected move, preserve its diagnostic trace, and make the result eligible for publication through the mailbox.

The outgoing transition from `DECISION`, rather than the state name itself, writes the response data and sets `READY` last. This ordering forms a small commit protocol: a reader never observes a ready result with partially written move data. It also provides a precise audit point: every external move can be associated with an accepted request sequence, a rules-valid candidate, its projected trace, and the policy comparison that selected it.

Deterministic tie-breaking is part of this boundary. When candidate values are equal, a canonical ordering—for example origin square followed by landing order—selects one move without hidden randomness. The result is repeatable across host and target adapters, which is essential for fixtures, replay, and comparison of future policy changes.

3.3 Why the separation matters

Without this split, a game program can blur three distinct questions: whether a move is legal, what its consequences are, and whether it should be exposed as the chosen action. The FSA assigns them to `GENERATE`, `ANTICIPATE`, and `SELECT/DECISION`, respectively. That allocation does not guarantee strong play; it makes the source of a choice testable. A future policy can become more sophisticated without changing the validity of the rules, the bounded nature of projection, or the mailbox commit discipline.

3.4 Portable boundary

The shared core is C++11 and avoids target-specific dependencies. Host emulation and target firmware are adapters around a stable board, mailbox, and agent interface. The intended target includes resource-constrained systems, but hardware measurements are not yet reported.

4 Validation methodology

Validation proceeds in layers:

1. Verify the 32-square board mapping, coordinate round trips, edge behavior, and invalid encodings with independent geometry fixtures.
2. Validate mailbox request parsing, sequencing, and error responses.
3. Fixture-test legal move generation and move application.
4. Add deterministic policy tests over fixed board positions.
5. Evaluate complete players against declared baselines using a documented, reproducible protocol.

Only the first three layers are partially implemented in this draft. The results section will remain intentionally incomplete until later layers and their measurements exist.

5 Current scope and limitations

The current rules implementation covers uncrowned-men quiet moves and single captures with compulsory-capture filtering. Promotion, king movement, multi-jumps, draw rules, policy, deeper projection, target adapters, and performance experiments remain future work. These limits are deliberate boundaries, not claims of completed functionality.

6 Related work and historical context

This section is reserved for a reviewed comparison with Checkers programs, finite-state game controllers, constrained game hardware, and period computer-player implementations. The project has an initial lead list, but citations will be added only after source verification. No novelty claim should be inferred from this placeholder section.

7 Evaluation plan

The completed paper will report:

- rules conformance against a documented position suite;
- determinism and protocol-error behavior;
- move quality against explicitly named baseline policies;
- memory, code size, and latency on each supported platform; and
- limitations, failure cases, and reproducibility instructions.

8 Conclusion

The project proposes a testable separation between game-control FSA, rules, and policy for portable deterministic Checkers play. The contribution of this draft is an accountable research direction and an implementation evidence trail. Any stronger claim awaits the remaining rules, experiments, and related-work review.